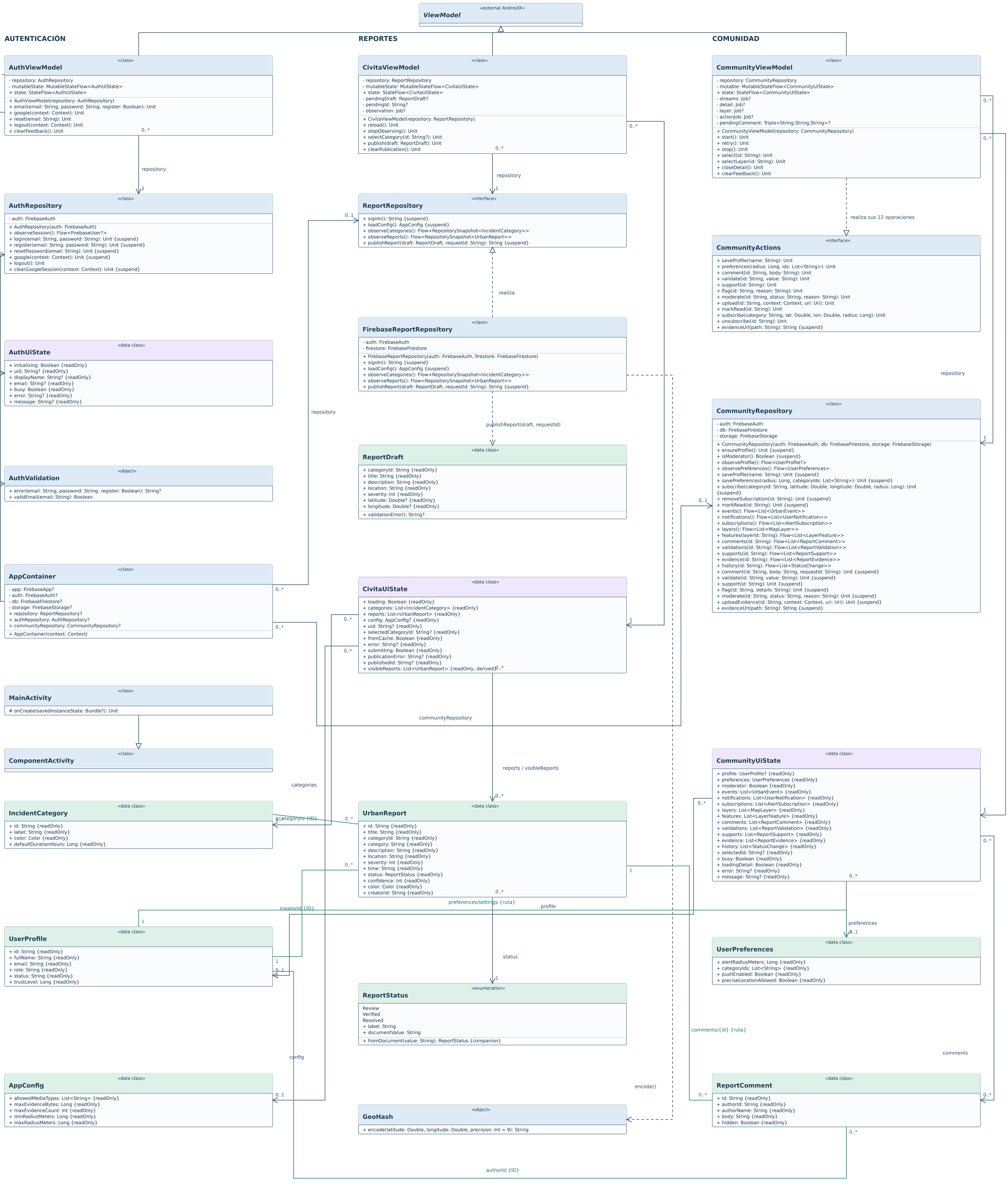
CIVITA · DIAGRAMA DE CLASES

MVVM por capas · Repositorios · Inyección manual · UML 2.5.1

Clases principales de autenticación, publicación y comunidad · Código Kotlin 1.1.742



NOTACIÓN Y ALCANCE

Triángulo hueco + línea continua: generalización. Los tres ViewModels heredan de AndroidX ViewModel.

Triángulo hueco + línea discontinua: realización. FirebaseReportRepository implementa ReportRepository;
CommunityViewModel implementa CommunityActions. Las operaciones del contrato no se repiten en su implementación.

Flecha abierta continua: asociación navegable. state.value es el objeto expuesto por StateFlow, no un miembro adicional.

Flecha abierta discontinua: dependencia de uso/parámetro. Línea verde: asociación lógica del dominio por ID de Firestore.

1 = exactamente uno; 0..1 = opcional; 0..* = varios. + público; - privado; # protegido; ? nullable; {readOnly} solo lectura; {suspend} coroutine.

No hay herencia entre las entidades Kotlin. No se afirma propiedad exclusiva ni borrado en cascada: no corresponden rombos.

Esta lámina muestra las clases principales. Tipos de detalle, fábricas y extensiones privadas están en CATALOGO-CODIGO.md.

Las pantallas Compose son funciones de nivel superior; no se convierten en clases ficticias. Los SDK se identifican por sus tipos.

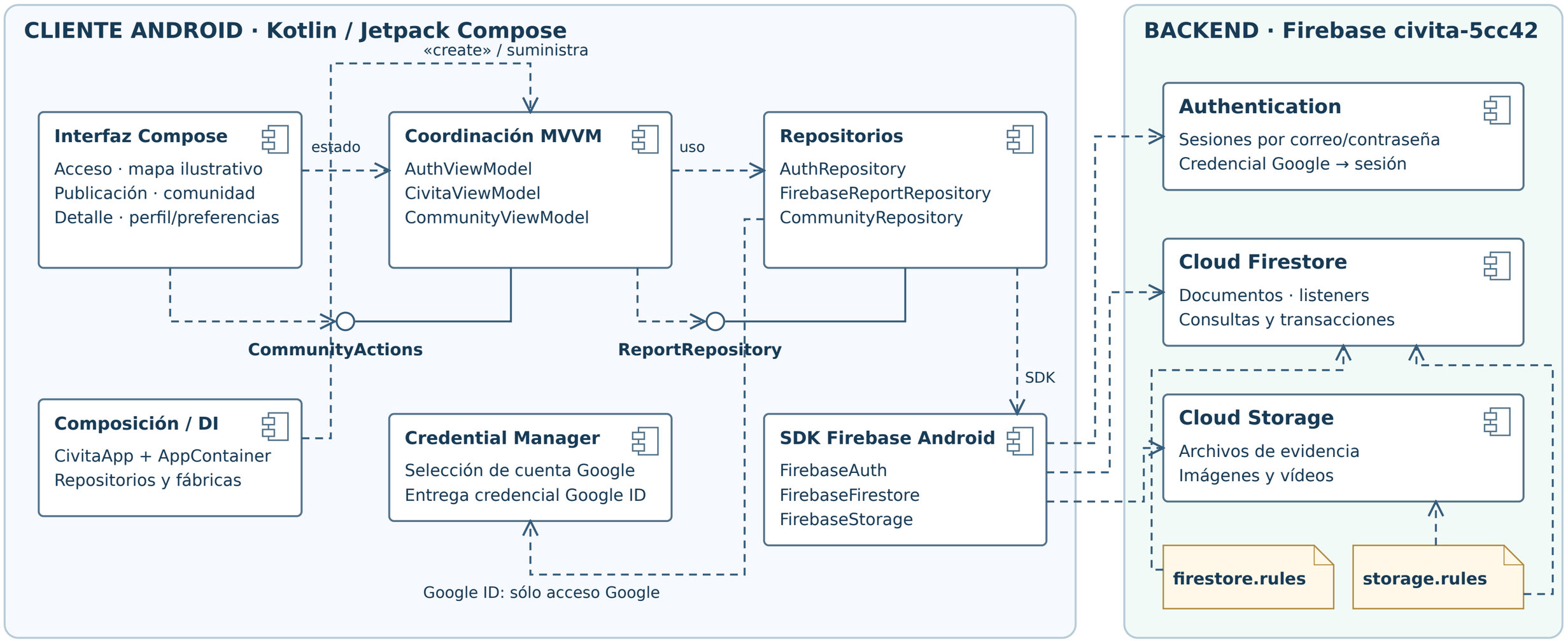
Las data classes generan constructor, copy, componentN, equals, hashCode y toString. Fuentes: app/src/main/java/.../civita.

Fuente de la arquitectura: CivitaApp.kt → ViewModels → repositorios → SDK Firebase. Diseño MVVM con backend administrado (BaaS).

Una página A0 vertical · PDF/SVG vectoriales: ampliar para revisar atributos, firmas y multiplicidades.

CIVITA · DIAGRAMA DE COMPONENTES

MVVM por capas + Repository · Cliente Android y backend administrado Firebase



LECTURA UML Y DECISIONES DE ARQUITECTURA

Dependencia «use»: el origen necesita el servicio o contrato de destino.

Interfaz proporcionada: ReportRepository por datos; CommunityActions por CommunityViewModel.

La UI observa StateFlow y envía acciones. Los repositorios encapsulan los SDK y las transacciones.

No hay API REST propia, microservicios ni capa de casos de uso. El cliente no publica reglas ni administra permisos.

Cloud Shell administra configuración y reglas por separado. Firestore autoriza documentos; Storage autoriza archivos.

Storage requiere bucket operativo y reglas. La simulación local es otra compilación y queda fuera de este despliegue.

Storage consulta autor y contador en Firestore para autorizar evidencias.

Fuente: implementación CIVITA 1.1.742 · Notación UML 2.5.1 · Una única lámina A3 horizontal